

JustPizza

Applicazione Android per l'ordinazione da una pizzeria locale

Yevhen Kukhar

Matricola 899720

Programmazione di Dispositivi Mobili

Università degli Studi di Milano-Bicocca

A.A. 2025/2026

Indice

1	Introduzione	2
2	Obiettivi e requisiti	2
2.1	Requisiti funzionali	2
2.2	Requisiti non funzionali (criteri d'esame)	2
3	Architettura	3
4	Modello di dominio	4
5	Storage locale (Room)	4
6	API esterne	4
6.1	Cloud Firestore — menu e ordini	5
6.2	Nominatim — consegna	5
7	Autenticazione	5
8	Offline-first e sincronizzazione cross-device	6
9	Interfaccia utente	6
10	Flusso utente (dall'accesso al pagamento)	6
11	Testing	7
12	Struttura del progetto e consegna	7
13	Conclusioni e sviluppi futuri	7

1 Introduzione

JustPizza è un'applicazione Android nativa, sviluppata in Java, che consente a un cliente di consultare il menu di una pizzeria locale, comporre un ordine e richiedere la consegna a domicilio entro un raggio massimo di 5 km.

L'idea nasce da un contesto reale: una piccola pizzeria non ha bisogno di un catalogo generico scaricato dall'Internet (ad esempio TheMealDB), ma delle proprie pizze, bibite e contorni, con prezzi e disponibilità gestiti dal locale. Per questo il catalogo vive su Cloud Firestore e viene copiato in un database Room sul dispositivo.

Il pagamento è **simulato**: non si integrano circuiti come Stripe o PayPal. L'esame valuta architettura, API esterne, autenticazione, persistenza e sincronizzazione, non un PSP reale. Dopo i controlli (rete, carrello non vuoto, indirizzo nel raggio) l'ordine viene salvato in locale e sul cloud e il carrello viene svuotato.

2 Obiettivi e requisiti

2.1 Requisiti funzionali

- Registrazione e accesso con email e password.
- Consultazione del menu (pizze, bibite, contorni) anche in assenza di rete, se il catalogo è già in cache.
- Aggiunta, visualizzazione, rimozione degli articoli nel carrello e calcolo del totale.
- Inserimento dell'indirizzo di consegna con suggerimenti e verifica del raggio di 5 km.
- Conferma dell'ordine (pagamento simulato) e consultazione dello storico.
- Visualizzazione del profilo e logout.

2.2 Requisiti non funzionali (criteri d'esame)

- **Architettura** a strati, ispirata alle linee guida Android e al progetto d'esempio del corso (*WorldNews*).
- **API esterne**: Firebase Authentication, Cloud Firestore, Nominatim (OpenStreetMap).
- **Autenticazione** con sessione persistente.
- **Offline-first**: l'UI legge sempre da Room.
- **Sincronizzazione cross-device** di menu e ordini tramite Firestore.
- **Storage locale** con Room.
- **Testing** JUnit sulla logica di dominio (carrello e distanza).
- **Layout** Material Design 3 e Navigation Component.
- **Documentazione** e repository GitHub secondo le indicazioni di consegna.

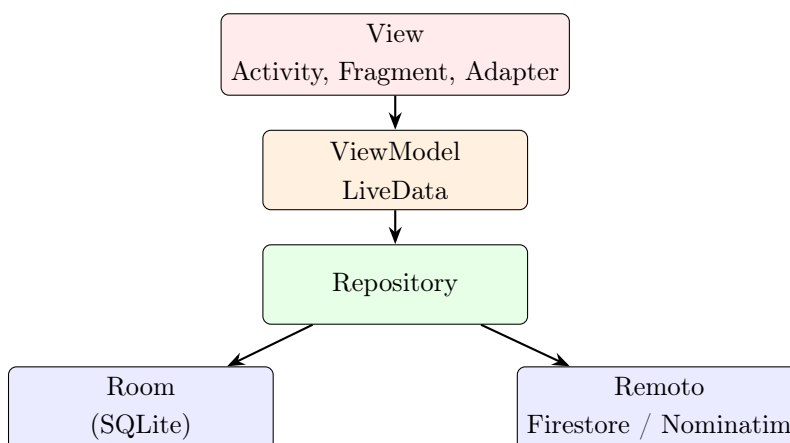
3 Architettura

L'applicazione segue il pattern **MVVM + Repository**, in linea con le guide Android e con il progetto d'esempio del corso (*WorldNews*).

Secondo le linee guida ufficiali i layer sono due, più il modello di dominio:

- **UI layer:** Activity, Fragment, Adapter e ViewModel. La View osserva LiveData; il ViewModel sopravvive alla rotazione e parla solo col Repository.
- **Data layer:** Repository, DAO Room e sorgenti remote (datasource). Il Repository è l'unico ingresso verso i dati, come `ArticleRepository` in *WorldNews*.
- **Model:** MenuItem, Pizza, Drink, Side, CartItem, Order.

Nel diagramma il ViewModel è disegnato sotto la View perché nel pattern MVVM è il mediatore: non è un terzo layer separato dal UI layer.



UI layer. `WelcomeActivity` è la launcher (login/register). `MainActivity` ospita la `BottomNavigationView` e, in `onCreate`, chiede al `PizzaViewModel` di aggiornare il menu da Firestore. I Fragment non usano le classi Room (`@Dao`, `AppDatabase`) né gli SDK Firebase: per menu, carrello e ordini passano dal ViewModel.

In alcuni punti autenticazione e geocoding la View chiama il data layer in modo diretto, per semplicità: `WelcomeActivity` e `ProfileFragment` usano `AuthRepository`; `CartFragment` usa `AuthRepository` e `DeliveryRemoteDataSource` (Nominatim). Login e registrazione passano invece da `AuthViewModel`.

Data layer. Ogni Repository coordina locale e remoto. Esempio: `PizzaRepository.getAllPizzas()` restituisce LiveData da `PizzaDAO`; `refreshMenu()` chiama `MenuRemoteDataSource` (collezione Firestore menu) e sovrascrive le tabelle Room su un `ExecutorService`.

Flusso della lista menu

1. `MainActivity` ottiene un `PizzaViewModel` di scope Activity e chiama `refreshMenu()` (Firestore → Room, se c'è rete; se fallisce resta la cache).
2. `MenuFragment` osserva `getAllPizzas()`, `getAllDrinks()` e `getAllSides()`.

3. Il `ViewModel` inoltra le richieste a `PizzaRepository`.
4. Il `Repository` espone i `LiveData` dei DAO. Quando `Room` viene aggiornata, la `RecyclerView` si ridisegna da sola, senza che il `Fragment` aspetti `Firestore`.

4 Modello di dominio

`MenuItem` è una classe astratta (Open/Closed Principle): i prodotti del menu condividono `id`, `name`, `price`, `imageUrl`, `description`. Aggiungere un nuovo tipo di prodotto non richiede di modificare il carrello.

- `Pizza` aggiunge `type` (*normale*, *calzone*, *teglia_famiglia*) e `ingredients`.
- `Drink` e `Side` specializzano il prodotto senza campi extra obbligatori.
- `CartItem` è un'entità `Room` a sé: non contiene un oggetto `MenuItem` (`Room` non serializza gerarchie polimorfiche in modo semplice), ma i campi denormalizzati `itemId`, `itemType`, `name`, `price`, `quantity`, `totalPrice`.
- `Order` rappresenta un ordine confermato: `userId`, `totalPrice`, `itemsSummary`, `orderDate`.

Gli identificatori delle voci di menu sono stringhe (`p001`, `d001`, ...) assegnate sul backend `Firestore`, così non collidono tra categorie. Il carrello usa invece un `cardId` intero autogenerated, perché la stessa pizza può essere aggiunta più volte.

5 Storage locale (Room)

Il database `SQLite` è gestito da `Room`, classe `AppDatabase`.

Entità: `Pizza`, `Drink`, `Side`, `CartItem`, `Order`.

DAO corrispondenti: `PizzaDAO`, `DrinkDAO`, `SideDAO`, `CartItemDAO`, `OrderDAO`.

Le letture esposte all'UI sono `LiveData`, in modo che un `INSERT` nel `Repository` aggiorni automaticamente `Fragment` e `Adapter`. Le scritture girano su un `ExecutorService` a singolo thread, per non bloccare il main thread (`Room` lo vieta di default).

Il campo `Date` di `Order` è convertito con `@TypeConverters(Converters.class)` da timestamp `Long`.

In fase di sviluppo è usato `fallbackToDestructiveMigration()`: cambiando schema il database locale viene ricreato. In un'app di produzione si definirebbero `Migration` esplicite; per un prototipo d'esame è accettabile.

6 API esterne

Le API esterne non servono a “scaricare pizze a caso”: il catalogo è della pizzeria. Servono tre integrazioni reali.

Servizio	Tecnologia	Ruolo
Firebase Auth	SDK Firebase	registrazione, login, sessione
Cloud Firestore	SDK Firebase	catalogo <code>menu</code> , ordini <code>orders</code>
Nominatim (OSM)	Retrofit + Gson	geocoding indirizzo di consegna

6.1 Cloud Firestore — menu e ordini

`MenuRemoteDataSource` legge la collezione `menu`. Ogni documento ha `category` (`pizza`, `drink`, `side`) e viene mappato sulla classe Java corretta. `PizzaRepository.refreshMenu()` scrive il risultato in Room.

`OrderRemoteDataSource` scrive e legge la collezione `orders`, filtrata per `userId` (UID Firebase). Così lo storico è per-utente e visibile da un altro telefono con lo stesso account.

6.2 Nominatim — consegna

`NominatimApi` è un'interfaccia Retrofit:

```
@GET("search")
Call<List<NominatimResult>> search(
    @Header("User-Agent") String userAgent,
    @Query("q") String query,
    @Query("format") String format,
    @Query("limit") int limit);
```

`DeliveryRemoteDataSource` (1) offre l'autocomplete dell'indirizzo con debounce di 500 ms; (2) geocodifica l'indirizzo scelto; (3) calcola la distanza dalla pizzeria con la formula di Haversine (`DistanceUtils.distanceKm`).

Coordinate della pizzeria (Monza): latitudine 45.59295, longitudine 9.26333. Raggio massimo: **5 km** (`Constants.MAX_DELIVERY_KM`). Nominatim richiede un `User-Agent` identificativo; è impostato nell'header della chiamata.

Il pagamento vero (carta, Google Pay) non è un'API di questo progetto. Si è scelto di non introdurre Stripe per non spostare lo scope dell'esame su chiavi, PCI e sandbox di pagamento.

7 Autenticazione

Firebase Authentication, provider **email/password**. Non è richiesta la verifica email: in emulatore Android Studio un ciclo di conferma via posta sarebbe scomodo e non è necessario per l'esame.

`AuthRepository` incapsula lo SDK:

- `register / login` con `OnCompleteListener`;
- `isLoggedIn()`, `getCurrentUser()`, `getEmail()`;
- `logout()` → `FirebaseAuth.signOut()`.

`WelcomeActivity` è la launcher. All'avvio: se esiste già una sessione Firebase si apre `MainActivity`; altrimenti si mostra `LoginFragment`. Dopo login/register si lancia `MainActivity` con flag `NEW_TASK` | `CLEAR_TASK`. Dal profilo, `logout` torna a `WelcomeActivity` con gli stessi flag, così il tasto Back non rientra nell'area autenticata.

Le credenziali non sono salvate in Room: le gestisce Firebase (token di sessione sul dispositivo). Gli ordini sul cloud sono legati all'uid dell'utente.

8 Offline-first e sincronizzazione cross-device

Offline-first. Ogni schermata di dati (menu, carrello, ordini) osserva Room. Senza rete l'utente vede l'ultimo menu scaricato, può modificare il carrello e consultare gli ordini già in cache. Non può invece pagare: `NetworkUtils.isOnline()` blocca la conferma, perché geocoding e upload Firestore richiedono Internet.

Sync del menu. Con rete, il repository scarica Firestore e sostituisce le tabelle locali. Un aggiornamento di prezzo fatto "lato pizzeria" sulla console Firebase arriva ai clienti al prossimo refresh, senza pubblicare una nuova versione dell'app.

Sync degli ordini. `OrderRepository.insertOrder` scrive prima in Room e poi su Firestore. `refreshOrders(userId)` riallinea lo storico. Due dispositivi con lo stesso account vedono gli stessi ordini (cross-device). Il carrello resta intenzionalmente locale: è un bozza d'acquisto del telefono, non un documento condiviso.

9 Interfaccia utente

Il design segue Material Design 3, come consigliato a lezione. Palette: crema `#F5EDE0` e bordeaux `#7A1F2B`.

- **Auth:** `TextInputLayout` per email e password (mostra/nascondi password), pulsanti Material, passaggio Login ↔ Register.
- **Main:** `BottomNavigationView` + `Navigation Component` (`mobile_navigation.xml`). Cinque destinazioni: Home, Menu, Carrello, Ordini, Profilo.
- **Menu:** `RecyclerView` con `ListAdapter/DiffUtil`; immagini con `Glide`; click → inserimento nel carrello e `Snackbar` ancorata sopra la bottom bar.
- **Carrello:** elenco quantità/prezzo, rimozione, totale osservato da `LiveData`, campo indirizzo `MaterialAutoCompleteTextView`, pulsanti Svuota e Paga.
- **Ordini:** lista degli ordini confermati (data, totale, riepilogo articoli).
- **Profilo:** email dell'account e pulsante Esci.

10 Flusso utente (dall'accesso al pagamento)

1. Avvio → `WelcomeActivity`. Login o registrazione.

2. **MainActivity**: si consulta il menu, si aggiungono pizze/bibite/contorni.
3. Nel carrello si rivede l'ordine, si imposta l'indirizzo (suggerimenti Nominatim).
4. **attemptPayment()**: (a) carrello non vuoto; (b) dispositivo online; (c) indirizzo risolto e distanza ≤ 5 km.
5. Si crea un **Order**, si salva (Room + Firestore), si svuota il carrello, si mostra conferma.
6. Lo storico è nella tab Ordini; dal Profilo si esce.

11 Testing

I test automatici sono JUnit su JVM (**app/src/test**), cioè sulla logica senza emulatore.

Classe	Cosa verifica
CartItemTest	<code>totalPrice = prezzo × quantità</code> ; con quantità 1 il totale coincide col prezzo unitario.
DistanceUtilsTest	stesso punto ≈ 0 km; Stazione di Monza ≤ 5 km (dentro raggio); Duomo di Milano > 5 km (fuori raggio).

Si eseguono da Android Studio (task Gradle **test**) oppure con `./gradlew test`. I test sulla distanza usano coordinate reali: proteggono la regola di business più delicata (“consegniamo solo vicino alla pizzeria”).

12 Struttura del progetto e consegna

Package principale: `it.unimib.justpizza`.

model/	<code>MenuItem, Pizza, Drink, Side, CartItem, Order</code>
database/	<code>AppDatabase, DAO</code>
datasource/	<code>Menu, Order, Delivery, Nominatim</code>
repository/	<code>Auth, Pizza, Drink, Side, Cart, Order</code>
viewmodel/	
ui/	<code>auth, home, menu, cart, orders, profile</code>
utils/	<code>Constants, Converters, DistanceUtils, NetworkUtils</code>

13 Conclusioni e sviluppi futuri

JustPizza copre i punti richiesti dall'esame: architettura MVVM con Repository, tre API esterne motivate, autenticazione Firebase, Room come source of truth, sync Firestore per menu e ordini, UI Material, test sulla logica di prezzo e di raggio.

Limiti consapevoli, da dichiarare in sede d'esame:

- pagamento simulato;

- nessuna app “admin” nativa (il menu si aggiorna dalla console Firebase);

Possibili estensioni: Google Sign-In, stato ordine (in preparazione / in consegna), notifiche, mappa OpenStreetMap, migrazioni Room versionate, test Instrumented sulle schermate.

Riferimenti

- Android Developers, *Guide to app architecture* (UI – Domain – Data).
- Progetto d’esempio del corso: L. Roviada, *WorldNews*, <https://github.com/lorenzoroviada/WorldNews>.
- Material Components for Android.
- Firebase Authentication e Cloud Firestore.
- Nominatim Usage Policy, OpenStreetMap.
- Repository del progetto: <https://github.com/JekaTSen/JustPizza>.